CSN 342
Deep Learning

1. School offering the course	School Of Computing
2. Course Code	CSF441
3. Course Title	**Deep Learning**
4. Credits (L:T:P:C)	2:0:1:3
5. Contact Hours (L:T:P)	2:0:2
6. Prerequisites (if any)	-
7. Course Basket	-

Faculty & Course coordinator :
Dr. Anushikha Singh

Textbook(s)

1. Deep Learning", Ian Goodfellow and Yoshua Bengio and Aaron Courville, MIT Press, 2016.

Reference Books

1. Neural Networks: A Systematic Introduction, Raúl Rojas, 1996
2. Pattern Recognition and Machine Learning, Christopher Bishop, 2007

SYLLABUS : Unit 4

UNIT 4: Convolutional Neural Networks and Recurrent Neural Networks

- Convolutional Neural Networks: The convolution operations: Filter/kernel, Stride, Padding, Pooling,
- CNN Architectures: LeNet, AlexNet.
- Transfer Learning and Pre-trained CNNs.
- Sequence learning problems, Recurrent Neural Networks, Back propagation through time,
- Long Short Term Memory, Gated Recurrent Units, Bidirectional LSTMs, Bidirectional GRUs.

Convolutional Neural Network

- Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset. For example visual datasets like images or videos where data patterns play an extensive role.
- ANN takes vectors as input so there is a need to convert images in the vector. Spatial information is lost in this process.
- CNN leading to sparse connections between input and output neurons: there wont be many weights such as ANN

What's the problem with simple NNs?

Regular artificial neural networks do not scale very well. For example, in CIFAR, a dataset that is commonly used for training computer vision models, the images are only of size 32x32 px and have 3 color channels. That means that a single fully-connected neuron in a first hidden layer of this neural network would have 32x32x3 = 3072 weights. It is still manageable. But now imagine a bigger image, for example, 300x300x3. It would have 270,000 weights (training of which demands so much computational power)!

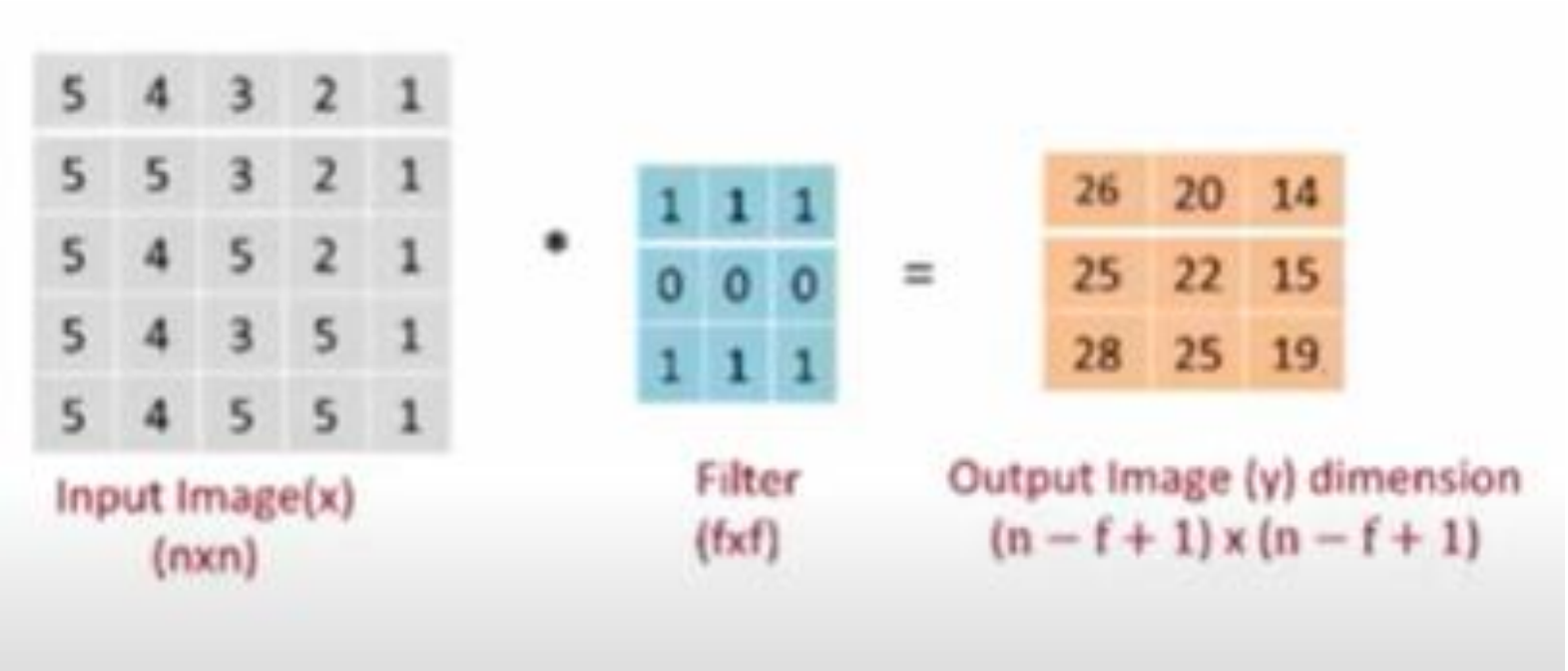
A huge neural network like that demands a lot of resources but even then remains prone to overfitting because the large number of parameters enable it to just memorize the dataset.

CNNs use parameter sharing. All neurons in a particular feature map share weights which makes the whole system less computationally intense.

How does a Convolutional Neural Network (CNN) work?

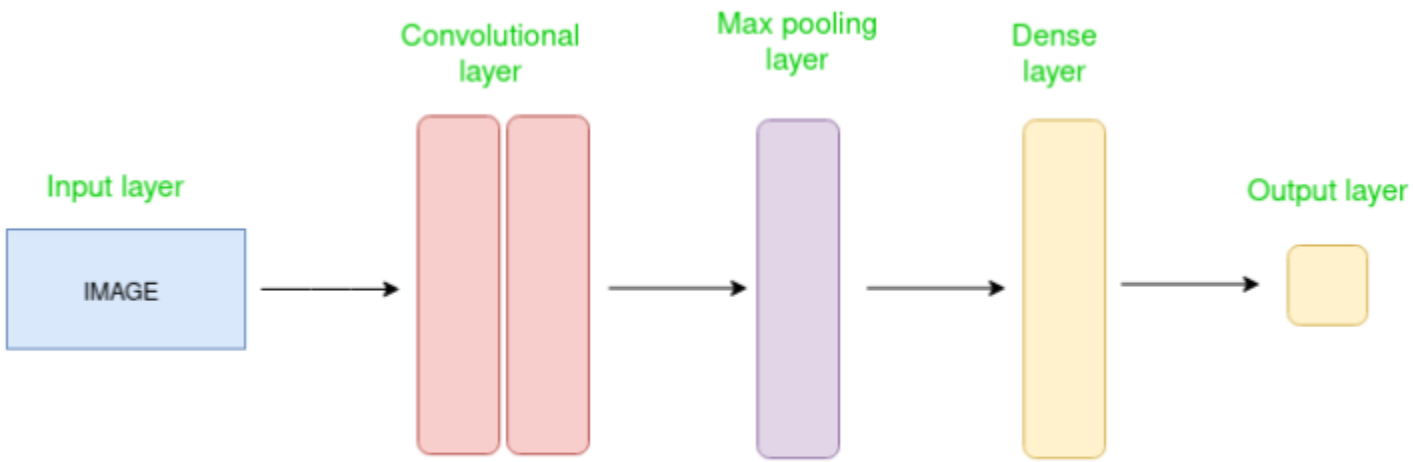
- A convolutional neural network, or ConvNet, is just a neural network that uses convolution.
- Convolution is a mathematical operation that allows the merging of two sets of information. In the case of CNN, convolution is applied to the input data to filter the information and produce a feature map.
- This filter is also called a kernel, or feature detector, and its dimensions can be, for example, 3x3. To perform convolution, the kernel goes over the input image, doing matrix multiplication element after element. The result for each receptive field (the area where convolution takes place) is written down in the feature map.
-

Convolution: Example



CNN Architecture

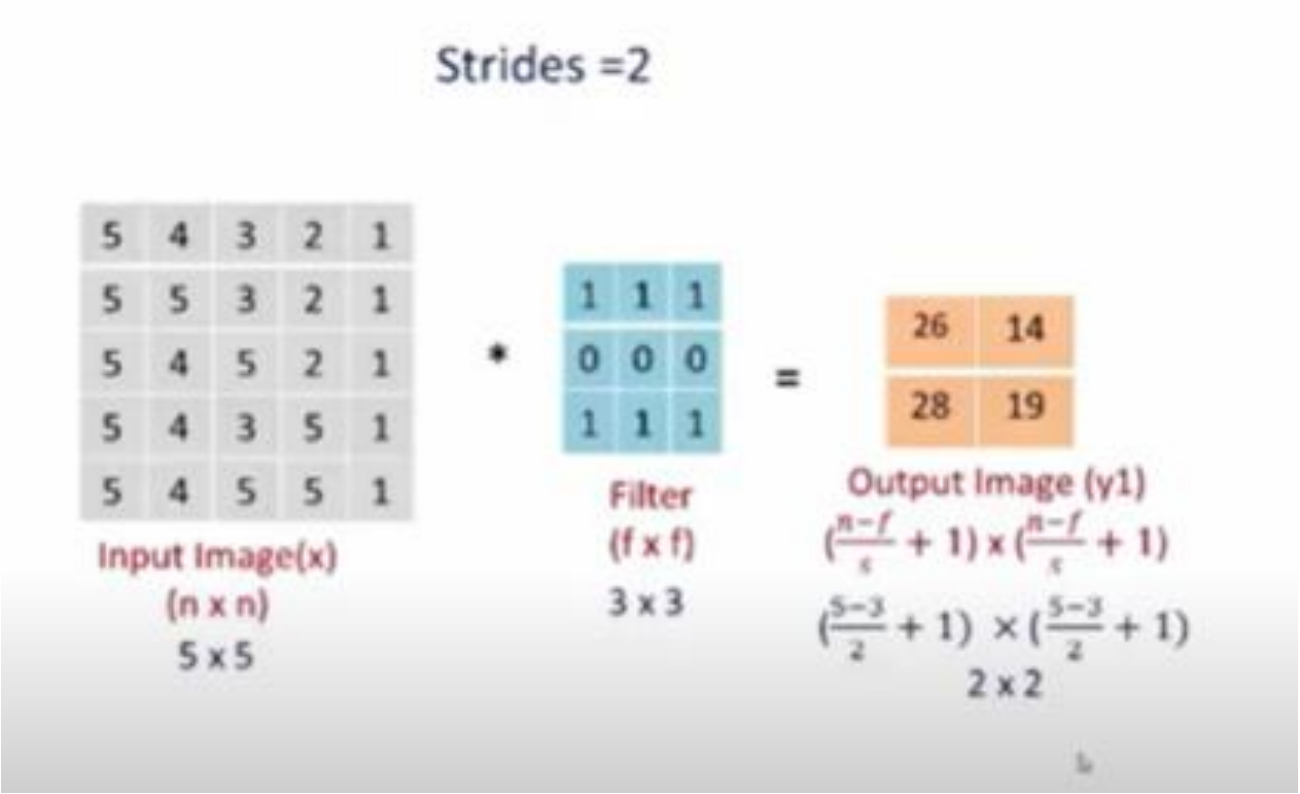
- Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.
- The Convolutional layer applies filters to the input image to extract features, the Pooling layer downsamples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.



Simple CNN architecture

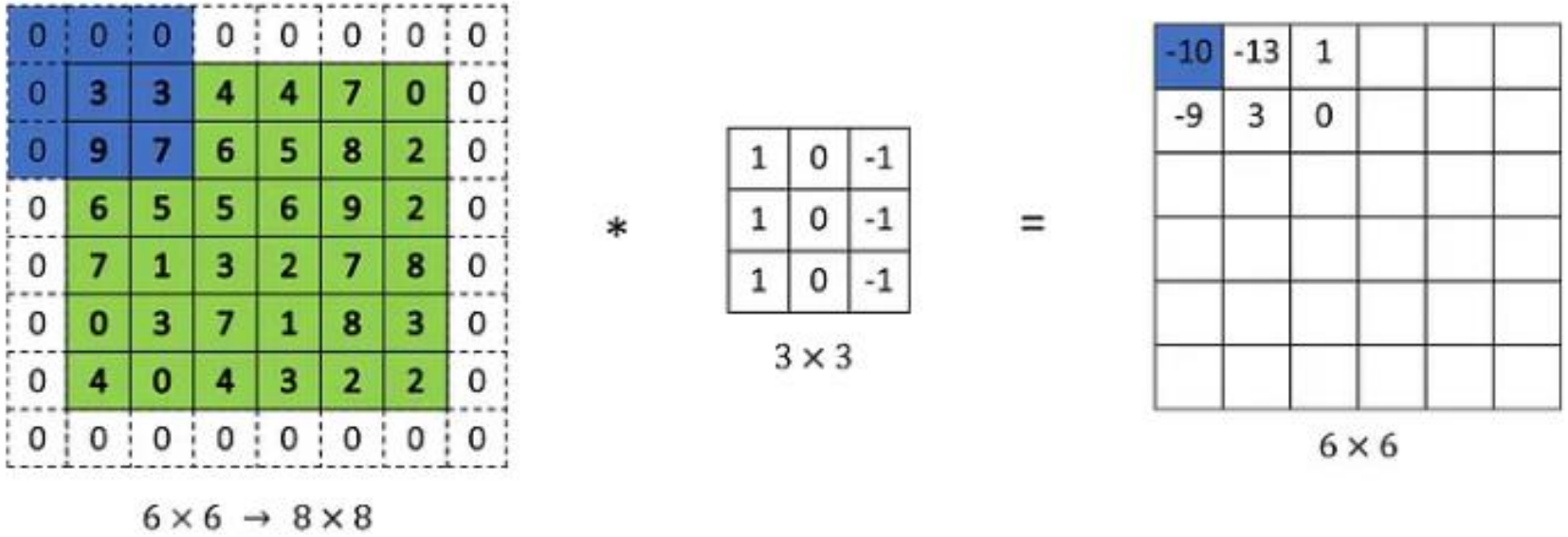
What is stride?

- The number of pixels we slide over the input image by the kernel is called a stride.



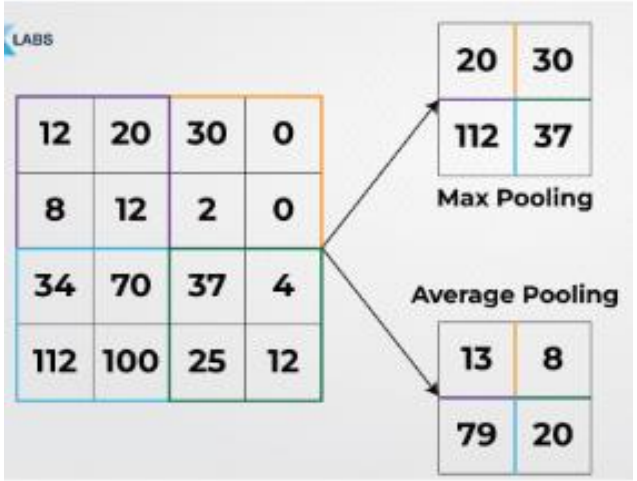
What is padding?

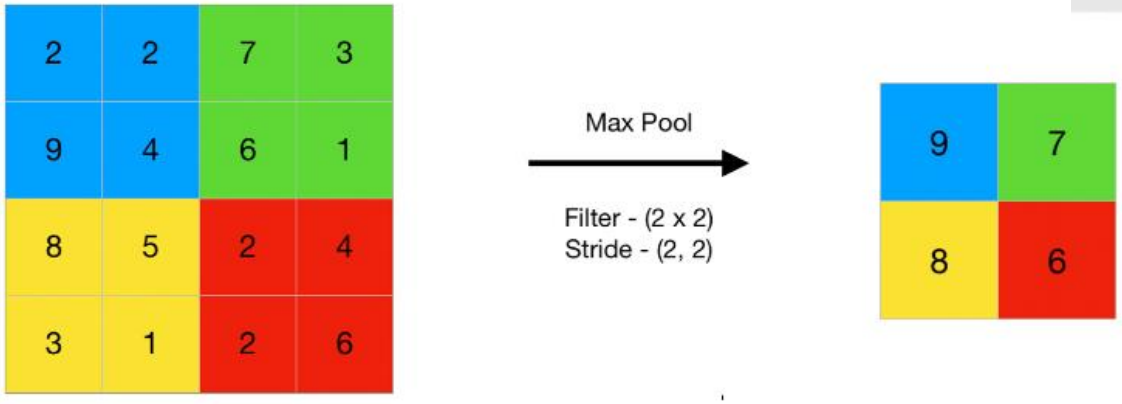
- At corners, we can not place the kernel so it is clear that the output of the convolution operation is smaller than the input image.
- ***What if we want the output as the same size as the input? or***
- ***what if the filter does not fit on the input image?***
- In that case, we add artificial padding of 0's around the image as per the required size and kernel size, shown in the below image which is also called **Zero-padding.**


0 0 0 0 0 0 0 0
0 3 3 4 4 7 0 0
0 9 7 6 5 8 2 0
0 6 5 5 6 9 2 0
0 7 1 3 2 7 8 0
0 0 3 7 1 8 3 0
0 4 0 4 3 2 2 0
0 0 0 0 0 0 0 0
6 × 6 → 8 × 8
*
1 0 -1
1 0 -1
1 0 -1
3 × 3
=
-10 -13 1
-9 3 0
6 × 6

Pooling

- A pooling layer is a component of a convolutional neural network (CNN) that reduces the spatial dimensions of feature maps and decreases the amount of data and parameters. It's also known as a downsampling layer.
- 2 Types: MAX Pooling and Average Pooling





Layers Used to Build ConvNets

- A complete Convolution Neural Networks architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.

  Types of layers: Input layer, convolutional layer, pooling layer, activation layer, flattening and fully connected layer

Datasets: Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.

- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2, 3×3, or 5×5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.

- **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**: max(0, x), **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.

- **Pooling layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.

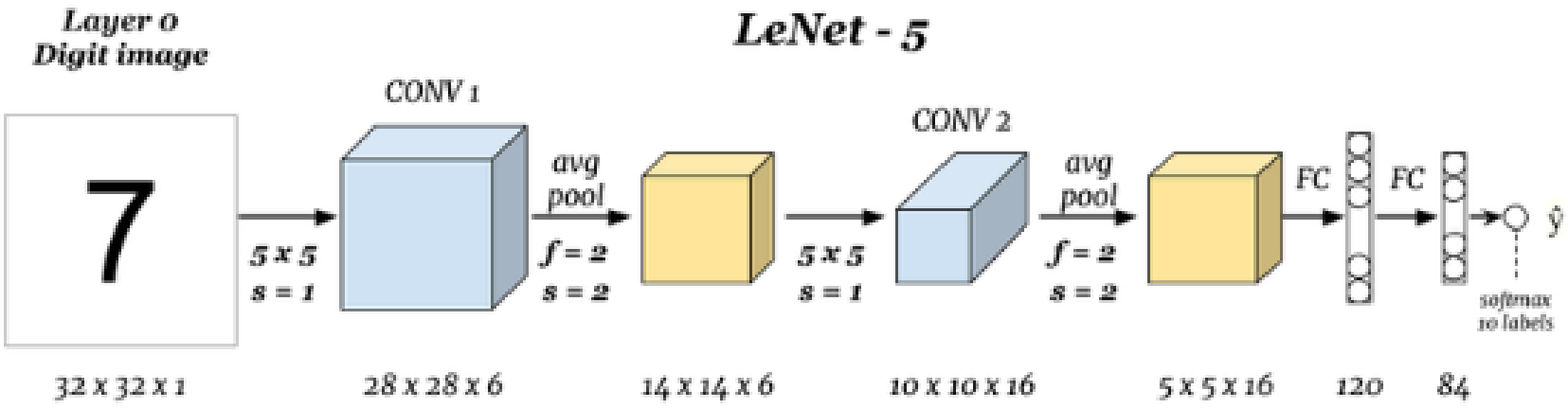
- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.

- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.

- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

Example: CNNs Architecture LeNet-5

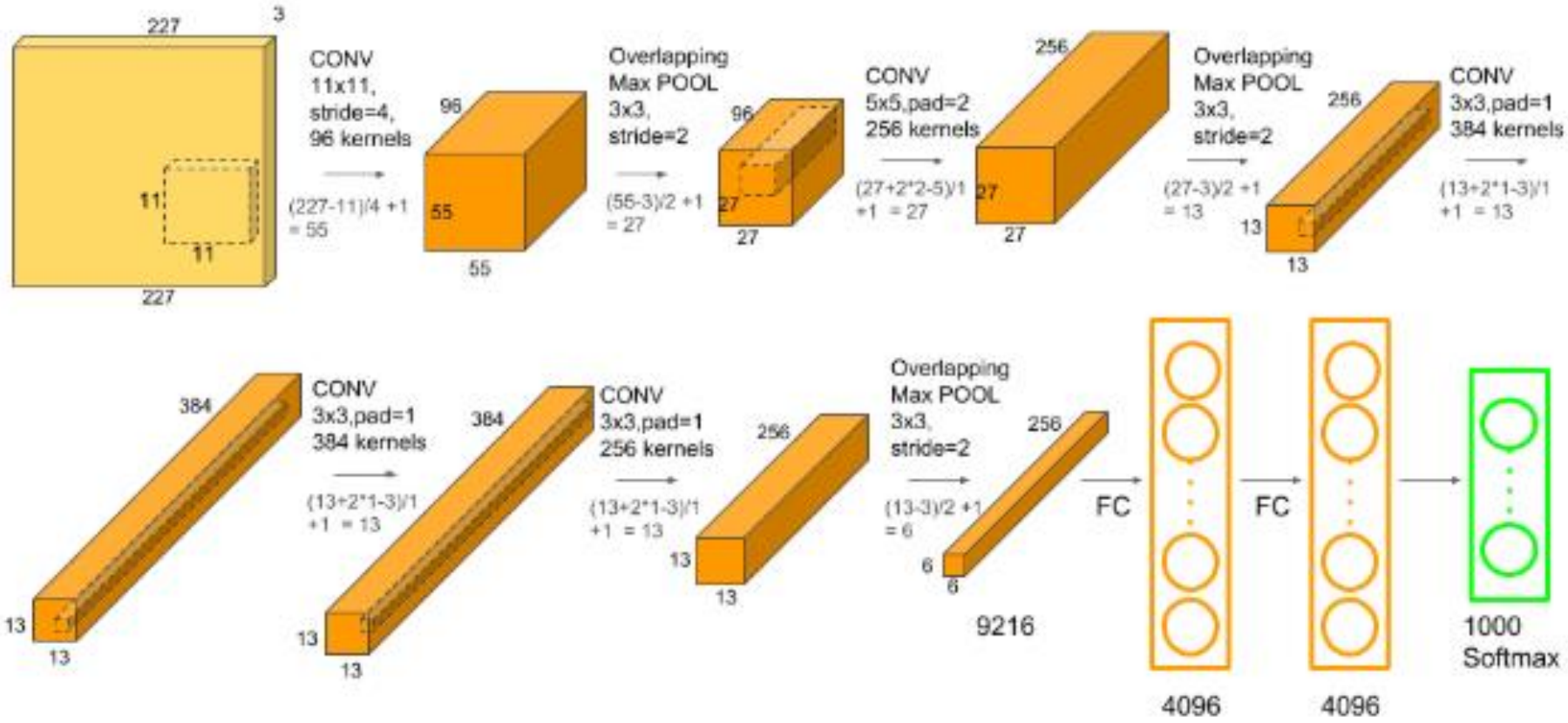
This is the architecture of LeNet-5 created by **Yann LeCun** in 1998 and widely used for written digits recognition (MNIST).



AlexNet

- AlexNet was designed by **Hinton**, winner of the ***2012 ImageNet competition***, and his student Alex Krizhevsky. It was also after that year that more and deeper neural networks were proposed, such as the excellent vgg, GoogleLeNet. Its official data model has an accuracy rate of **57.1%** and top 1-5 reaches **80.2%**. This is already quite outstanding for traditional machine learning classification algorithms.

Architecture



Loss Function used in LeNet and AlexNet

Cross-Entropy Loss (Categorical Cross-Entropy)

For a **multi-class classification problem**, the loss function used is:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

where:

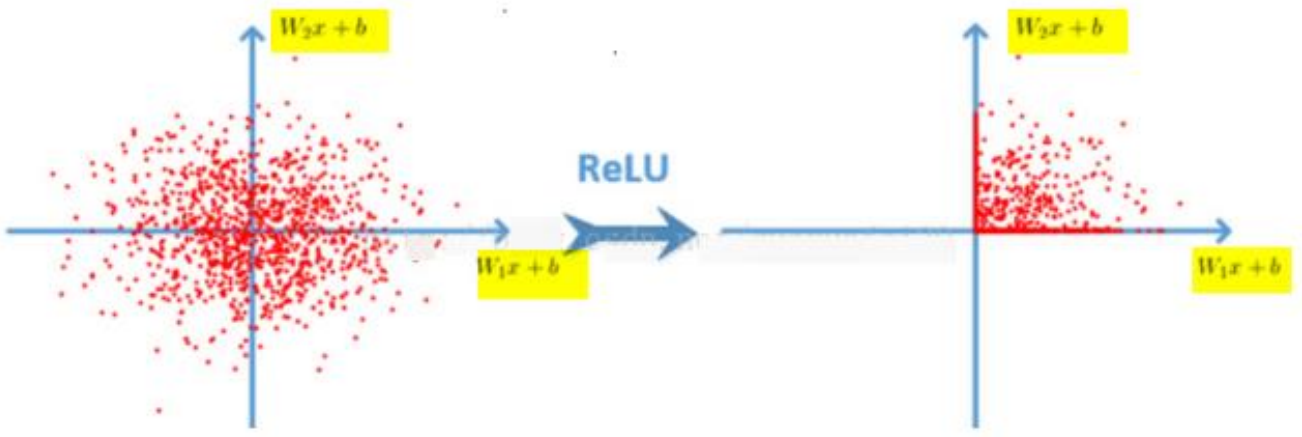
- y_i is the true label (one-hot encoded for multiple classes).
- $\hat{y}_i$ is the predicted probability for class i.
- N is the total number of classes.

Why does AlexNet achieve better results?

1. Relu activation function is used:

- ReLU-based deep convolutional networks are trained several times faster than tanh and sigmoid- based networks. The following figure shows the number of iterations for a four-layer convolutional network based on CIFAR-10 that reached 25% training error in tanh and ReLU:

- Relu function: f (x) = max (0, x)



2. Standardization (Local Response Normalization):

- After using ReLU f (x) = max (0, x), you will find that the value after the activation function has no range like the tanh and sigmoid functions, so a normalization will usually be done after ReLU, and the LRU is a steady proposal (Not sure here, it should be proposed?) One method in neuroscience is called "Lateral inhibition", which talks about the effect of active neurons on its surrounding neurons.

3. Dropout:

- Dropout is also a concept often said, which can effectively prevent overfitting of neural networks. Compared to the general linear model, a regular method is used to prevent the model from overfitting. In the neural network, Dropout is implemented by modifying the structure of the neural network itself.

Numerical: LeNet

Assume we have a **small dataset** where the model classifies images into three classes:

- **Class 0**
- **Class 1**
- **Class 2**

Now, suppose the model predicts probabilities for a given input image as:

Class	Predicted Probability ($\hat{y}_i$)	Actual Label (y_i)
0	0.7	1 (Correct)
1	0.2	0
2	0.1	0

Calculate the loss for this Model

Solution

The formula for **cross-entropy loss** is:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

Since the actual class is **0**, the loss simplifies to:

$$L = -\log(0.7)$$

Approximating $\log(0.7) \approx -0.357$,

$$L = -(-0.357) = 0.357$$

Question: Consider a CNN that takes a grayscale image of size 32×32 as input. The first convolutional layer has 8 filters, each with a kernel size of 3×3, a stride of 1, and no padding.

1. Calculate the output dimensions (height and width) after this convolutional layer.
2. If the layer is followed by a max-pooling layer with a 2×2 kernel and a stride of 2, what will be the output dimensions after the max-pooling layer?
3. Assuming the output of the max-pooling layer is flattened and passed to a fully connected layer with 64 neurons, determine the number of weights required for this layer.

Solution Guide:

1. **Convolutional Layer Output Dimensions:**
   - Input image dimensions: 32×32
   - Filter size: 3×3
   - Stride: 1
   - Padding: 0

Using the formula for output dimension:

$$\text{Output Height/Width} = \frac{\text{Input Height/Width} - \text{Filter Size}}{\text{Stride}} + 1$$

Substitute values for height and width:

$$\frac{32 - 3}{1} + 1 = 30$$

So, the output dimensions after the convolutional layer are 30×30, with 8 feature maps.

2. **Max-Pooling Layer Output Dimensions:**
   - Max-pooling kernel size: 2×2
   - Stride: 2

   Output dimensions:

$$\frac{30}{2} = 15$$

   So, the output dimensions after the max-pooling layer are 15×15, with 8 feature maps.

3. **Fully Connected Layer Weights:**

   - Flattened input size: $15 \times 15 \times 8 = 1800$
   - Fully connected layer with 64 neurons requires:

$$1800 \times 64 = 115,200 \text{ weights}$$

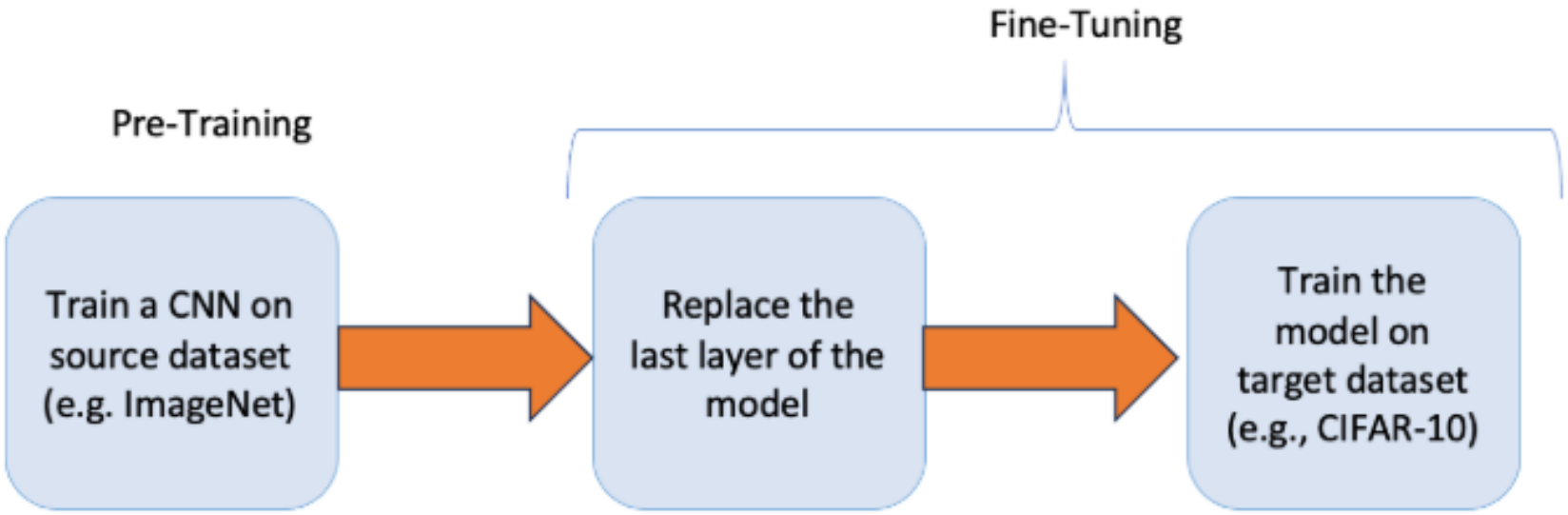
Thus, the fully connected layer requires **115,200 weights.**

Transfer Learning

- Transfer learning is a technique in deep learning where pre-trained models trained on large-scale datasets are used to solve new tasks with limited labeled data.
- Transfer learning is a machine learning technique where a model trained on one task is repurposed as the foundation for a second task. This approach is beneficial when the second task is related to the first or when data for the second task is limited.
- Using learned features from the initial task, the model can adapt more efficiently to the new task, accelerating learning and improving performance. Transfer learning also reduces the risk of overfitting, as the model already incorporates generalizable features useful for the second task.

Working of Transfer Learning

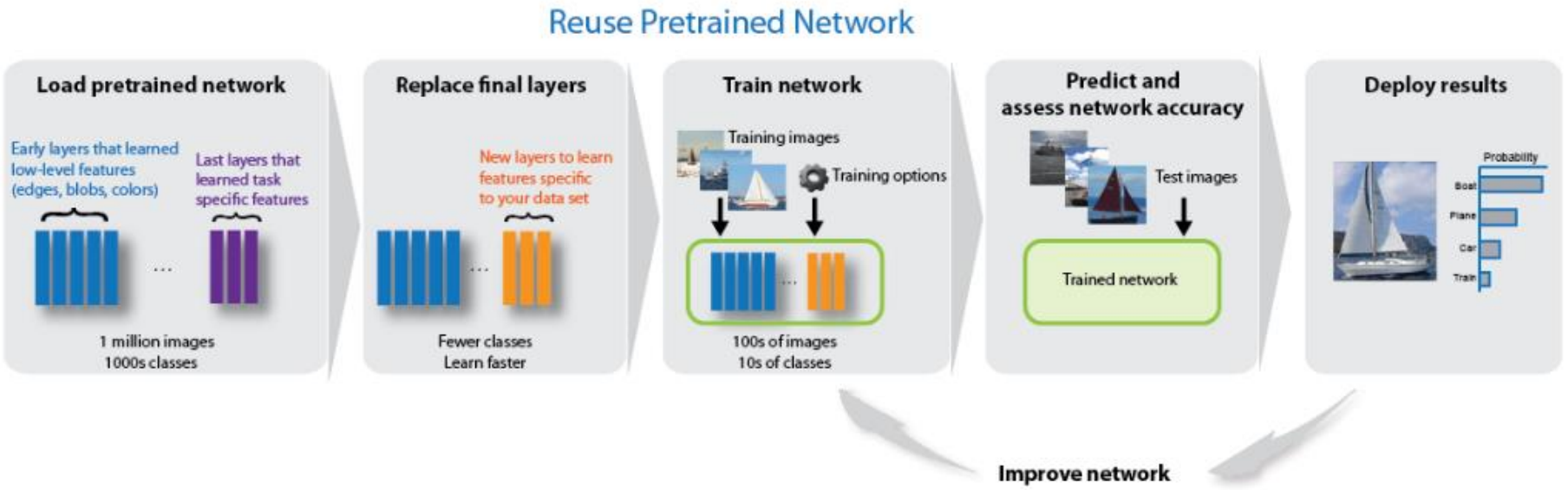
- Transfer learning involves a structured process to use existing knowledge from a pre-trained model for new tasks:
- **Pre-trained Model:** Start with a model already trained on a large dataset for a specific task. This pre-trained model has learned general features and patterns that are relevant across related tasks.
- **Base Model:** This pre-trained model, known as the base model, includes layers that have processed data to learn hierarchical representations, capturing low-level to complex features.
- **Transfer Layers:** Identify layers within the base model that hold generic information applicable to both the original and new tasks. These layers often near the top of the network capture broad, reusable features.
- **Fine-tuning:** Fine-tune these selected layers with data from the new task. This process helps retain the pre-trained knowledge while adjusting parameters to meet the specific requirements of the new task, improving accuracy and adaptability.


Fine-Tuning
Pre-Training
Train a CNN on source dataset (e.g. ImageNet)
Replace the last layer of the model
Train the model on target dataset (e.g., CIFAR-10)

Transfer Learning Using AlexNet

- This example shows how to fine-tune a pretrained AlexNet convolutional neural network to perform classification on a new collection of images.
- AlexNet has been trained on over a million images and can classify images into 1000 object categories (such as keyboard, coffee mug, pencil, and many animals). The network has learned rich feature representations for a wide range of images. The network takes an image as input and outputs a label for the object in the image together with the probabilities for each of the object categories.

- Transfer learning is commonly used in deep learning applications. You can take a pretrained network and use it as a starting point to learn a new task. Fine-tuning a network with transfer learning is usually much faster and easier than training a network with randomly initialized weights from scratch. You can quickly transfer learned features to a new task using a smaller number of training images.



Benifts of Transfer Learning

- Transfer learning offers solutions to key challenges like:
- **Limited Data:** Acquiring extensive labelled data is often challenging and costly. Transfer learning enables us to use pre-trained models, reducing the dependency on large datasets.
- **Enhanced Performance:** Starting with a pre-trained model which has already learned from substantial data allows for faster and more accurate results on new tasks ideal for applications needing high accuracy and efficiency.
- **Time and Cost Efficiency:** Transfer learning shortens training time and conserves resources by utilizing existing models hence eliminating the need for training from scratch.
- **Adaptability:** Models trained on one task can be fine-tuned for related tasks making transfer learning versatile for various applications from image recognition to natural language processing.

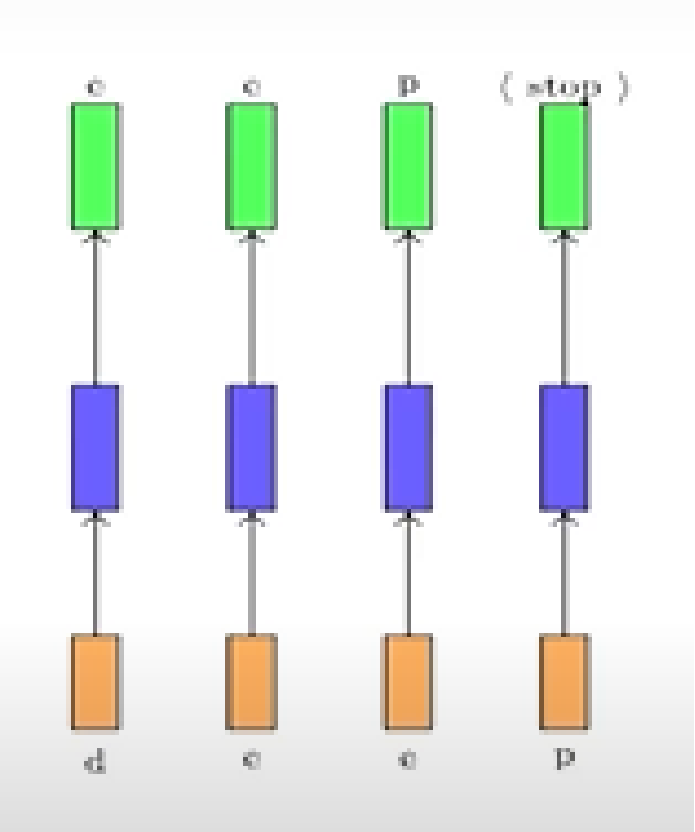
Recurrent Neural Network (RNN)

Sequence learning problems

Feedforward neural network and Convolutional Neural Network share common characteristics:

- **Input data size is fixed** i.e. the size of input data does not change for different training instances or different test instances.
- **Inputs to the network are independent of past or future inputs.** e.g. — for an image recognition network, if you pass an image of cat or of a dog, network treats them independently and detects the class image belongs to. There is no dependency on what the network has seen before or after.

- In many applications these properties are not true. Let's consider the **task of character prediction,** in which the output is predicted at every time step. The inputs (characters) are not of fixed size and the next character prediction is related to the characters that have already been seen by network.
- These kind of problems are known as ***sequence learning problems***, where there is a sequence of inputs and network produces outputs. Each input correspond to one time step.



1. *Prediction of a word "deep"*
2. ***Each network is performing the same task***
3. *Network is*

Input layer ⟶ Hidden Layer ⟶ Output layer

<u>Sequence learning problem:</u>

- Input size is not fixed
- Inputs are dependent on each other

Recurrent Neural Networks

- *Model to deal with sequences*

- **Requirement for the model:**

1. Account for dependence between inputs

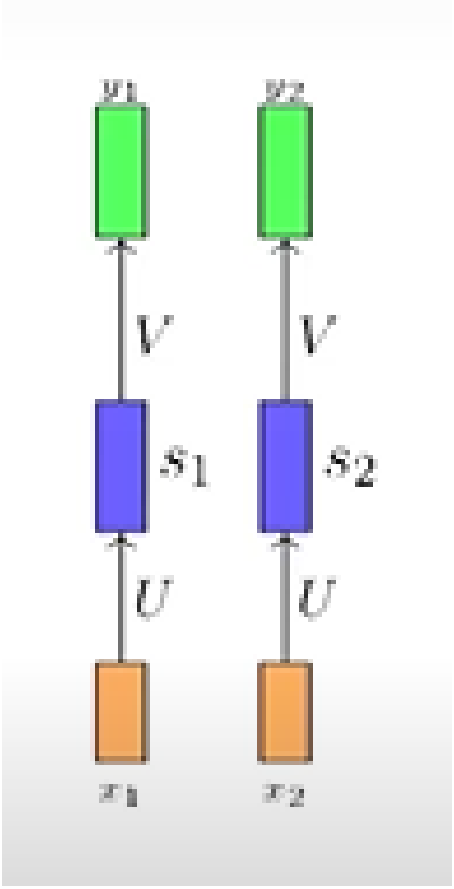
(the output actually depends on multiple inputs and not just a single input.)

2. Account for variable no of inputs

(because a video could be 300 seconds, it could be 20 seconds, 25 seconds; a sentence could be of arbitrary lines and so on.)

3. Function executed at each time step is same

(every time step they are trying to do the same activity)



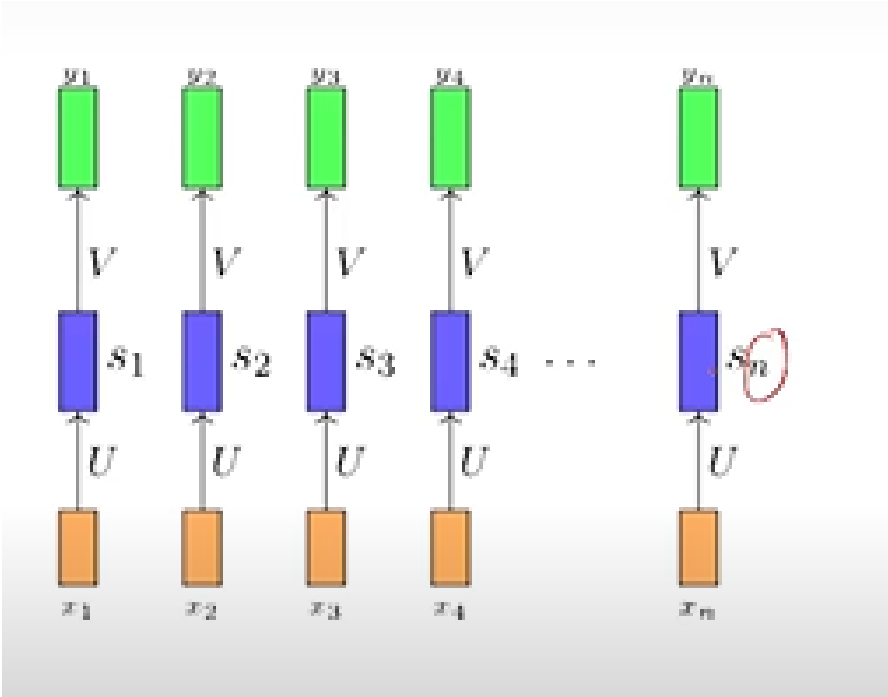
Function executed at each time step

$$s_i = \sigma(Ux_i + b)$$
$$y_i = \sigma(Vs_i + c)$$
$$i = \text{timestep}$$

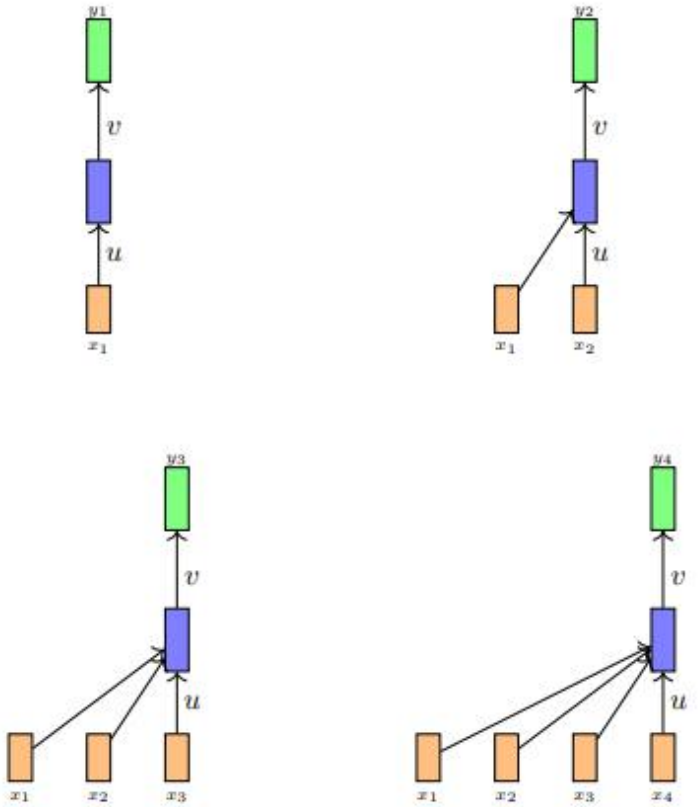
- Since we want the same function to be executed at each timestep we should share the same network (i.e., same parameters at each timestep)

Same parameters of the network at each time step

1. This parameter sharing also ensures that the network becomes agnostic to the length of the input.
2. Since we are simply going to compute the same function (with same parameters) at each time step the number of time steps does not matter. Because now whether I have a word which has 10 characters or 20 characters it does not matter, because at every time step I am going to execute the same function. That is why it is important that at every time step we have the same function.
3. We just create multiple copies of the network and execute them at each time step

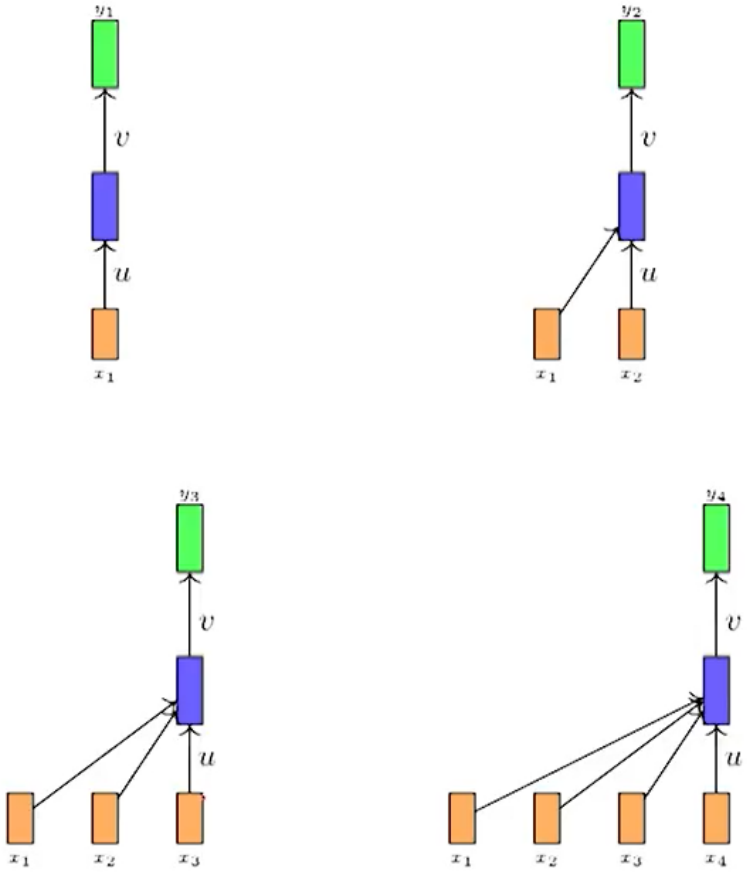


How we deal with dependency between inputs



- How do we account for dependence between inputs ?
- Let us first see an infeasible way of doing this
- At each timestep we will feed all the previous inputs to the network
- Is this okay ?
- No, it violates the other two items on our wishlist
- How ? Let us see

Is this a good way ?



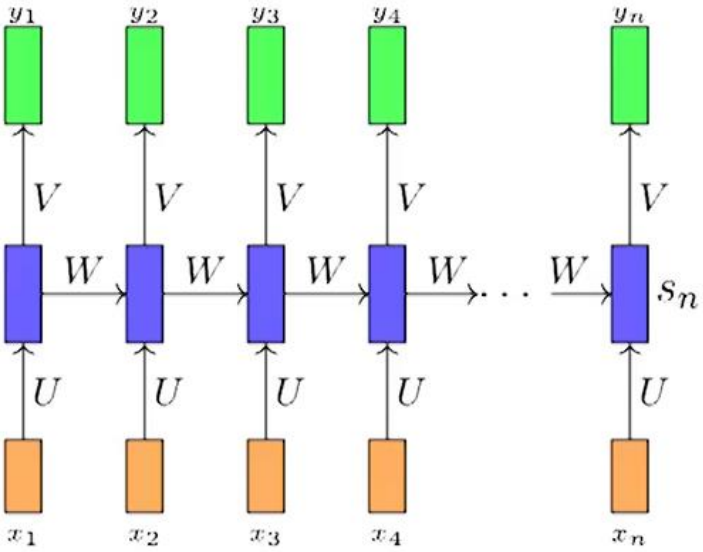
- First, the function being computed at each time-step now is different

$$y_1 = f_1(x_1)$$
$$y_2 = f_2(x_1, x_2)$$
$$y_3 = f_3(x_1, x_2, x_3)$$

- The network is now sensitive to the length of the sequence
- For example a sequence of length 10 will require $f_1, \ldots, f_{10}$ whereas a sequence of length 100 will require $f_1, \ldots, f_{100}$

Not a possible Solution

Final Solution (Architecture of RNN)

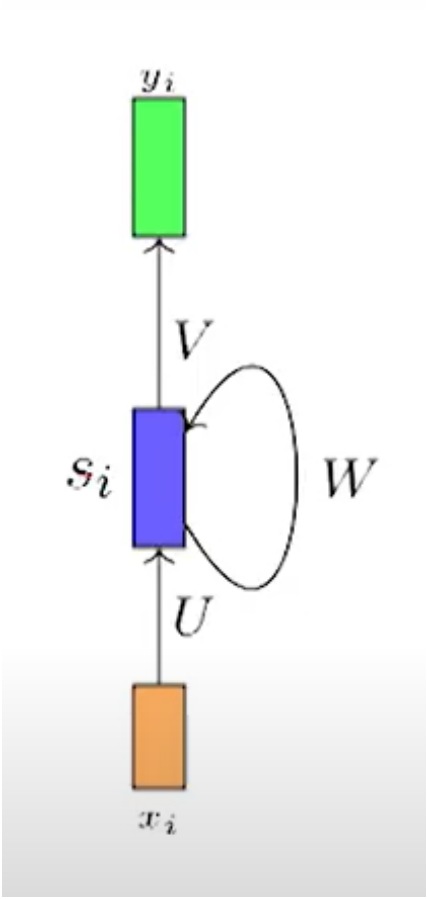


- The solution is to add a recurrent connection in the network,

$$s_i = \sigma(Ux + Ws_{i-1} + b)$$
$$y_i = \sigma(Vs_i + c)$$
$$or$$
$$y_i = f(x_i, s_i, W, U, V)$$

- s_i is the state of the network at timestep i
- The parameters are W, U, V which are shared across timesteps
- The same network (and parameters) can be used to compute $y_1, y_2, \ldots, y_{10}$ or y_{100}

We can write it in a simple way: Recurrent Neural Network(RNN)



RNN meets all the requirements for handling sequence learning problems.

- The same function is executed at each time step.
- It processes inputs of arbitrary length since the function is applied repeatedly at each time step.
- It ensures that the output depends on previous inputs.

Architecture of a Simple RNN

- In a simple RNN, the data flows through a series of steps, each one taking the output of the previous step as part of its input. Consider the following architecture for a basic RNN:
- **Input Layer**: The input at each time step is passed into the network.
- **Hidden Layer**: The hidden state is updated based on the current input and the previous hidden state.
- **Output Layer**: The output at each time step is computed based on the current hidden state.

Why we can not use standard back propagation for Training of RNN ?

Standard backpropagation cannot be directly used for training Recurrent Neural Networks (RNNs) due to the **temporal nature of RNNs** and their unique structure. Instead, we use **Backpropagation Through Time (BPTT)**, which is an extension of standard backpropagation adapted for sequential data. The main reasons why standard backpropagation is not suitable for RNNs are:

1. Sequential Dependencies and Unfolding Over Time

- Unlike feedforward networks where inputs are independent, RNNs process sequential data where each output depends on previous hidden states.
- To compute gradients, we need to **unfold** the RNN over time, treating it as a deep computational graph, which requires modifications to standard backpropagation.

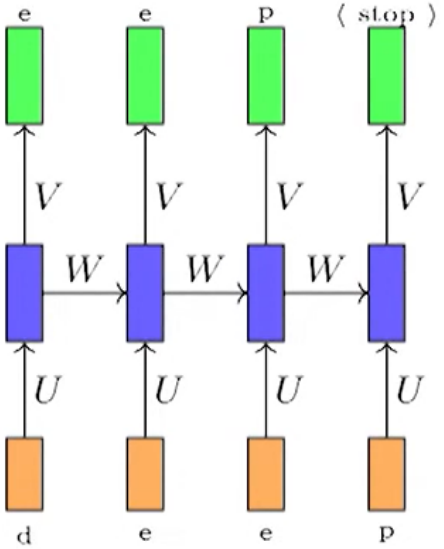
2. Weight Sharing Across Time Steps

- RNNs use the **same weights at each time step**, unlike feedforward networks where different layers have independent parameters.
- Backpropagation needs to account for this weight sharing, which is handled by **Backpropagation Through Time (BPTT)**.

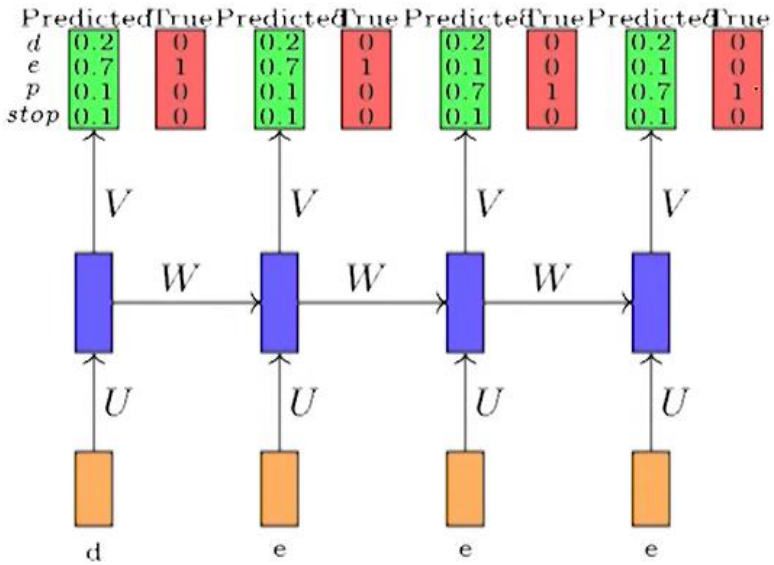
3. Handling Long-Term Dependencies

- Due to the vanishing gradient problem, standard backpropagation struggles to retain information over long sequences.

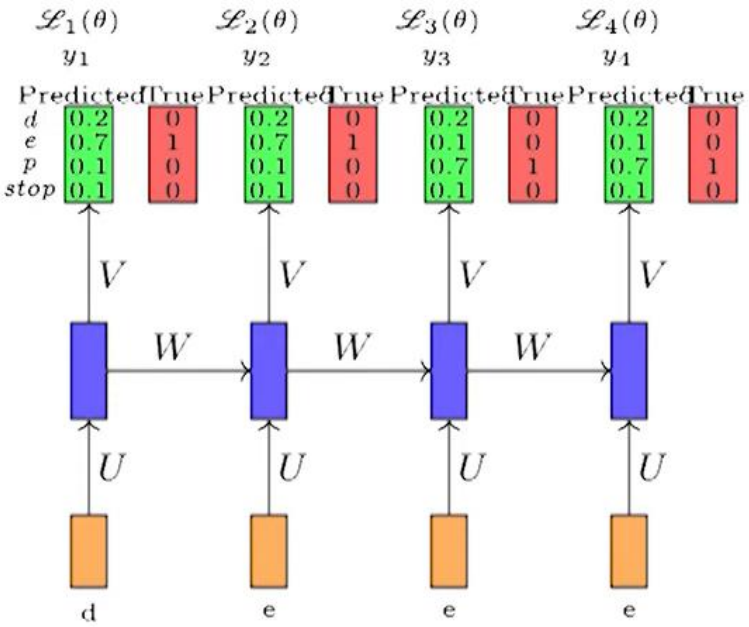
Back propagation through time: Training of RNN



- Suppose we consider our task of auto-completion (predicting the next character)
- For simplicity we assume that there are only 4 characters in our vocabulary (d,e,p, <stop>)
- At each timestep we want to predict one of these 4 characters
- What is a suitable output function for this task ? **(softmax)**
- What is a suitable loss function for this task ? **(cross entropy)**



- Suppose we initialize U, V, W randomly and the network predicts the probabilities as shown
- And the true probabilities are as shown
- We need to answer two questions
- What is the total loss made by the model ?
- How do we backpropagate this loss and update the parameters ($\theta = \{U, V, W\}$) of the network ?



- The total loss is simply the sum of the loss over all time-steps

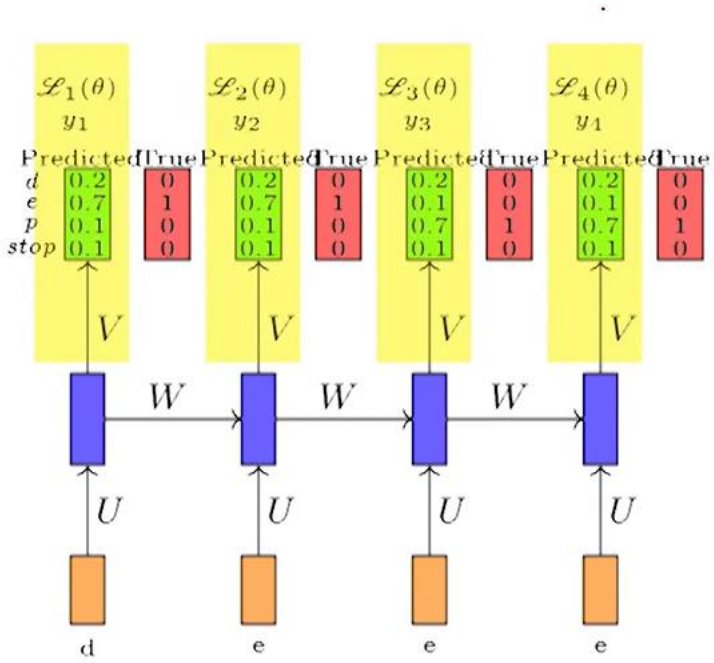
$$\mathscr{L}(\theta) = \sum_{t=1}^{T} \mathscr{L}_t(\theta)$$

$$\mathscr{L}_t(\theta) = -log(y_{tc})$$ Cross entropy loss

y_{tc} = predicted probability of true character at time-step i

T = number of timesteps

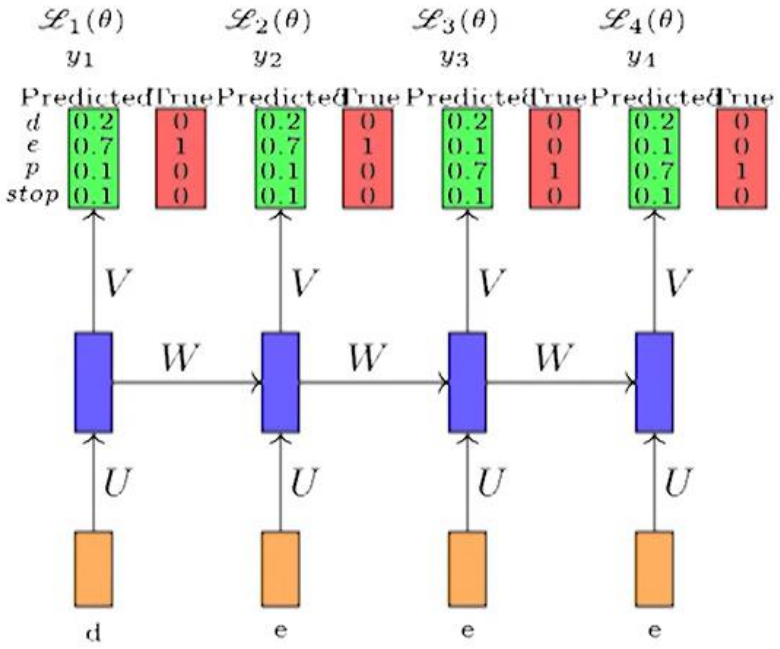
- For backpropagation we need to compute the gradients w.r.t. W, U, V
- Let us see how to do that



- Let us consider $\frac{\partial \mathscr{L}(\theta)}{\partial V}$ (V is a matrix so ideally we should write $\nabla_v \mathscr{L}(\theta)$)

$$\frac{\partial \mathscr{L}(\theta)}{\partial V} = \sum_{t=1}^{T} \frac{\partial \mathscr{L}_t(\theta)}{\partial V}$$

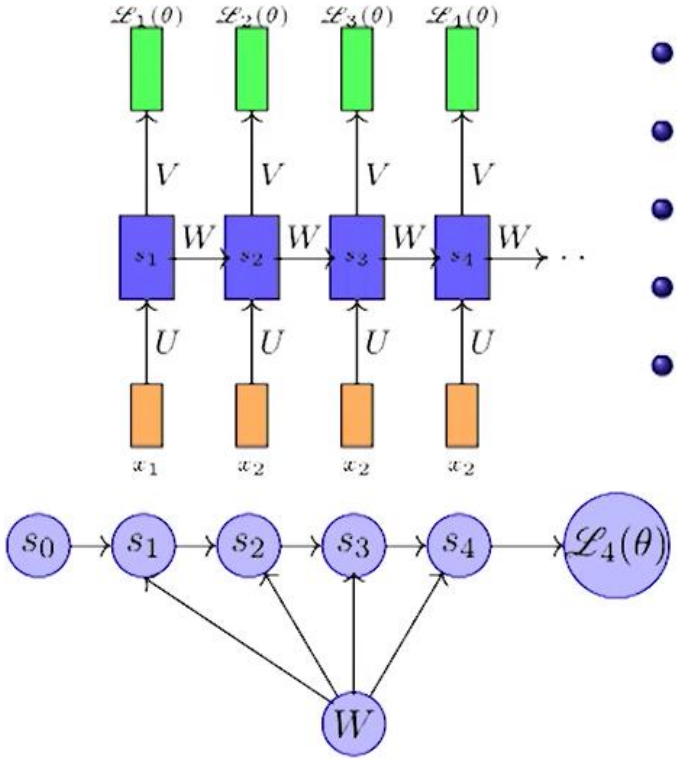
- Each term is the summation is simply the derivative of the loss w.r.t. the weights in the output layer
- We have already seen how to do this when we studied backpropagation



- Let us consider the derivative $\frac{\partial \mathscr{L}(\theta)}{\partial W}$

$$\frac{\partial \mathscr{L}(\theta)}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathscr{L}_t(\theta)}{\partial W}$$

- By the chain rule of derivatives we know that $\frac{\partial \mathscr{L}_t(\theta)}{\partial W}$ is obtained by summing gradients along all the paths from $\mathscr{L}_t(\theta)$ to W
- What are the paths connecting $\mathscr{L}_t(\theta)$ to W ?
- Let us see this by considering $\mathscr{L}_4(\theta)$

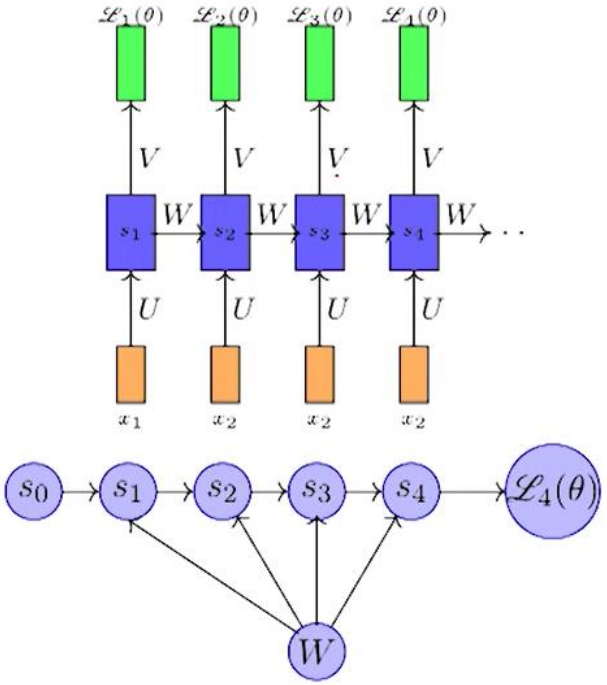


- $\mathscr{L}_4(\theta)$ depends on s_4
- s_4 in turn depends on s_3 and W
- s_3 in turn depends on s_2 and W
- s_2 in turn depends on s_1 and W
- s_1 in turn depends on s_0 and W

where s_0 is a constant starting state.

Ordered Network

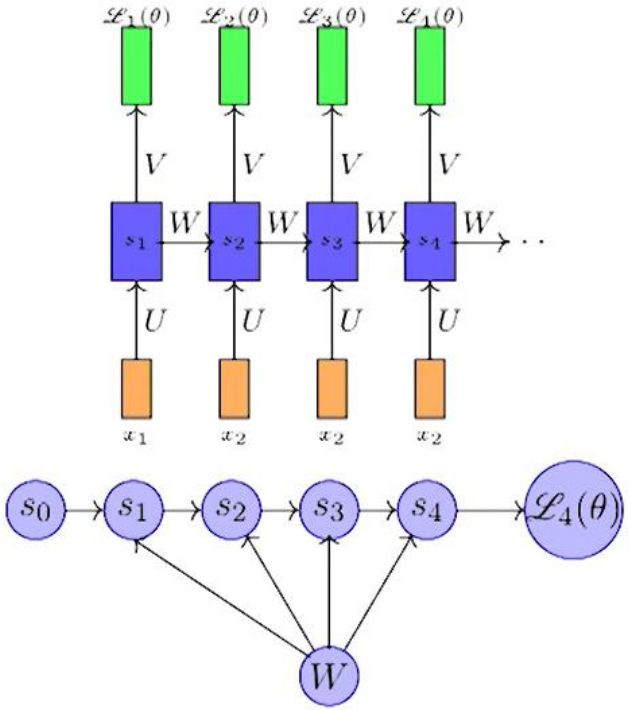
An **Ordered Network in RNN** refers to the structured sequence processing capability of **Recurrent Neural Networks (RNNs)**, where data is processed in a step-by-step fashion, preserving the temporal or sequential dependencies in the input.



- What we have here is an ordered network
- In an ordered network each state variable is computed one at a time in a specified order (first s_1, then s_2 and so on)
- Now we have

$$\frac{\partial \mathscr{L}_4(\theta)}{\partial W} = \frac{\partial \mathscr{L}_4(\theta)}{\partial s_4} \frac{\partial s_4}{\partial W}$$

- We have already seen how to compute $\frac{\partial \mathscr{L}_4(\theta)}{\partial s_4}$ when we studied backprop
- But how do we compute $\frac{\partial s_4}{\partial W}$



- Recall that

$$s_4 = \sigma(W s_3 + b)$$

- In such an ordered network, we can't compute $\frac{\partial s_4}{\partial W}$ by simply treating s_3 as a constant (because it also depends on W)
- In such networks the total derivative $\frac{\partial s_4}{\partial W}$ has two parts
- **Explicit** : $\frac{\partial^+ s_4}{\partial W}$, treating all other inputs as constant
- **Implicit** : Summing over all indirect paths from s_4 to W

$$\frac{\partial s_4}{\partial W} = \underbrace{\frac{\partial^+ s_4}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial W}}_{\text{implicit}}$$

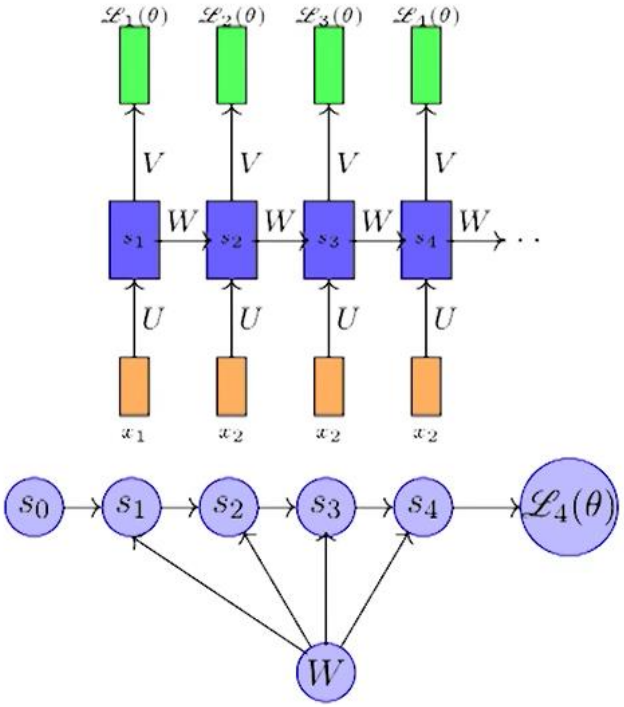
$$= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\Big[\underbrace{\frac{\partial^+ s_3}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial W}}_{\text{implicit}}\Big]$$

$$= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\Big[\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_2}{\partial s_1}\frac{\partial s_1}{\partial W}\Big]$$

$$= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial s_1}\Big[\frac{\partial^+ s_1}{\partial W}\Big]$$

For simplicity we will short-circuit some of the paths

$$\frac{\partial s_4}{\partial W} = \frac{\partial s_4}{\partial s_4}\frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_2}\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_4}{\partial s_1}\frac{\partial^+ s_1}{\partial W} = \sum_{k=1}^{4}\frac{\partial s_4}{\partial s_k}\frac{\partial^+ s_k}{\partial W}$$



- Finally we have

$$\frac{\partial \mathscr{L}_4(\theta)}{\partial W} = \frac{\partial \mathscr{L}_4 \theta}{\partial s_4} \frac{\partial s_4}{\partial W}$$

$$\frac{\partial s_4}{\partial W} = \sum_{k=1}^{4} \frac{\partial s_4}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$$

$$\therefore \frac{\partial \mathscr{L}_t(\theta)}{\partial W} = \frac{\partial \mathscr{L}_t(\theta)}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$$

- This algorithm is called backpropagation through time (BPTT) as we backpropagate over all previous time steps

Applications of RNN

- **Speech Recognition** (e.g., Siri, Google Assistant)
- **Language Modeling** (e.g., Text Prediction, Chatbots)
- **Time-Series Forecasting** (e.g., Stock Market, Weather Prediction)
- **Machine Translation** (e.g., Google Translate)

Issues of Standard RNNs:

1. ***Vanishing Gradient***
2. ***Exploding Gradient***

- **Vanishing Gradient:**

  Vanishing gradient problem is a phenomenon that occurs during the training of deep neural networks, where the gradients that are used to update the network become extremely small or "vanish" as they are backpropogated from the output layers to the earlier layers.

The vanishing gradient problem can cause: Slow convergence, The network getting stuck in low minima, Impaired learning of deep representations, and The training process to completely stall

Some techniques that can help alleviate the vanishing gradient problem include:

1. **ReLU activation function**: Can be used to replace the sigmoid activation function
2. **Batch normalization**: Normalizes the inputs to each layer within a mini-batch
3. **Proper weight initialization**: Using Xavier or He initialization ensures that the gradients do not vanish or explode
4. **Residual networks (ResNets)**: Use skip connections to allow the gradients to bypass some layers and directly flow to deeper layers

Exploding Gradient

The exploding gradient problem is a common issue in deep neural networks that occurs when the gradient increases significantly during training. This can cause the network to become unstable, making it unable to learn from training data.

- **Causes of Exploding Gradients:** The root cause of exploding gradients can often be traced back to the network architecture and the choice of activation functions. In deep networks, when multiple layers have weights greater than 1, the gradients can grow exponentially as they propagate back through the network during training. This is exacerbated when using activation functions with outputs that are not bounded, such as the hyperbolic tangent or the sigmoid function.
- Another contributing factor is the initialization of the network's weights. If the initial weights are too large, even a small gradient can be amplified through the layers, leading to very large updates during training

Several strategies can be employed to mitigate the exploding gradient problem:

1. **Gradient Clipping:** This technique involves setting a threshold value, and if the gradient exceeds this threshold, it is scaled down to keep it within a manageable range. This prevents any single update from being too large.
2. **Weight Initialization:** Using a proper weight initialization strategy, such as Xavier or He initialization, can help prevent gradients from becoming too large at the start of training.
3. **Use of Batch Normalization:** Batch normalization can help maintain the output of each layer within a certain range, reducing the risk of exploding gradients.
4. **Change of Network Architecture:** Simplifying the network architecture or using architectures that are less prone to exploding gradients, such as those with skip connections like ResNet, can be effective.
5. **Proper Activation Functions:** Using activation functions that are less likely to produce large gradients, such as the ReLU function and its variants, can help control the gradient's magnitude.